Bitácora S07: Mexi Banco

De un monolito a servicios que se encuentran solos

Sistemas Distribuidos (Semestre 2027-1, FI-UNAM)

Victor Emiliano Calderón Gutiérrez

No. de cuenta 423097331

17 de octubre de 2026

Entorno

Hora: 03:04

Para este laboratorio trabajé sobre la rama `lab/03-gateway-discovery-balanceo` del repositorio https://github.com/OscarRuiz21/mexi-banco y, tal como lo pide la guía, creé mi propia rama de trabajo `trabajo-s07`. Todo lo corrí en Windows 11 con Docker 29.8.2, usando Git Bash como terminal (y PowerShell en algunas de las capturas).

Un problema antes de empezar. Antes siquiera de llegar al primer paso, la construcción me falló con `/bin/sh: ./mvnw: not found`. Investigando un poco, el problema era que en Windows Git tiene activado `core.autocrlf=true`, por lo que al clonar convirtió los finales de línea de `mvnw` y de `demo-v06.sh` a CRLF. Es decir, dentro del contenedor (que es Linux) la primera línea queda como `#!/bin/sh\r`, y ese intérprete no existe. Para arreglarlo desactivé esa opción solo en ese clon con `git config core.autocrlf false`, volví a sacar los archivos (`git rm -cached -r .` y `git reset -hard`) y con eso ya pudo construir sin problema.

1. La historia: partimos el banco

Hora: 03:10

Tal como lo vimos en la clase, en el monolito una transferencia era simplemente una llamada a método dentro del mismo proceso: `TransferenciaService` le pedía a `CuentaService` que hiciera el cargo y el abono, y los dos quedaban dentro del mismo `@Transactional` sobre la misma base. Por lo que si el abono fallaba, la base también deshacía el cargo, es decir, o pasaba todo o no pasaba nada.

El problema es que al partirlo en cinco servicios, cada uno con su propia base, `cuentas` dejó de ser un objeto en memoria y ahora es un cliente que hace una petición HTTP a otro proceso. Lo que yo entiendo que se rompió aquí es la atomicidad: el cargo se confirma en la base de `cuenta` en cuanto esta responde, y si después el abono falla (porque se corta la red, `cuenta` no contesta o hay un timeout), ya no existe un `@Transactional` que lo deshaga, ya que cada servicio solo puede confirmar lo suyo. Entonces el dinero puede quedar cargado en el origen y nunca llegar al destino.

Además de eso, apareció otro problema, que es saber dónde está cada servicio. Si abrimos el `docker-compose.yml`, vemos que `CUENTA_URL` está escrito a mano como `http://cuenta:8080`, y de ahí salen los tres problemas que se trabajan en este laboratorio: cómo se encuentran los servicios entre sí, cuántas puertas ve el cliente y cómo repartir el trabajo entre varias copias de un mismo servicio.

2. El directorio: que cada servicio se anote solo

2.0 · Antes de tocar nada

Hora: 03:21

Para empezar, levanté el proyecto tal como venía. Con `docker compose ps` podemos ver seis contenedores (la base y los cinco servicios), cada uno con su puerto publicado en mi máquina, del 8081 al 8085. Después, en el navegador, al abrir `localhost:8081/cuentas/000` es `cuenta` quien contesta directamente con un 404 en JSON (`"No existe la CLABE 000"`). Mientras tanto, `localhost:8761` no carga y marca `ERR_CONNECTION_REFUSED`, ya que nadie está escuchando en ese puerto porque todavía no hay directorio.

```
PS C:\Users\battl\Documents\sd-2027-1\entregas\calderon_victor\s07\mexi-banco> docker compose ps
NAME                                IMAGE                               COMMAND                  SERVICE         CREATED         STATUS                   PORTS
mexi-banco-v06-cuenta-1             mexi-banco-v06-cuenta               "java -jar app.jar"      cuenta          4 minutes ago   Up 4 minutes (healthy)   0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp
mexi-banco-v06-db-1                 postgres:17-alpine                  "docker-entrypoint.s…"   db              4 minutes ago   Up 4 minutes (healthy)   5432/tcp
mexi-banco-v06-movimiento-1         mexi-banco-v06-movimiento           "java -jar app.jar"      movimiento      4 minutes ago   Up 4 minutes (healthy)   0.0.0.0:8082->8080/tcp, [::]:8082->8080/tcp
mexi-banco-v06-notificacion-1       mexi-banco-v06-notificacion         "java -jar app.jar"      notificacion    4 minutes ago   Up 4 minutes (healthy)   0.0.0.0:8084->8080/tcp, [::]:8084->8080/tcp
mexi-banco-v06-spei-1               mexi-banco-v06-spei                 "java -jar app.jar"      spei            4 minutes ago   Up 4 minutes (healthy)   0.0.0.0:8085->8080/tcp, [::]:8085->8080/tcp
mexi-banco-v06-transferencia-1      mexi-banco-v06-transferencia        "java -jar app.jar"      transferencia   4 minutes ago   Up 4 minutes (healthy)   0.0.0.0:8083->8080/tcp, [::]:8083->8080/tcp
PS C:\Users\battl\Documents\sd-2027-1\entregas\calderon_victor\s07\mexi-banco>
```

Figura 1: `docker compose ps` antes de los cambios: seis contenedores `healthy`, cada servicio con su puerto.

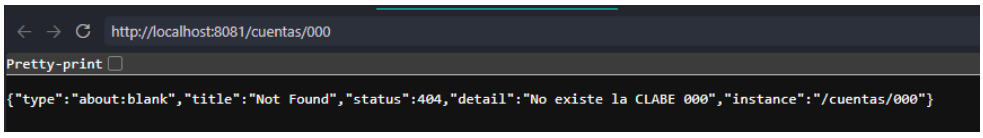


Figura 2: `localhost:8081/cuentas/000`: `cuenta` contesta directo con un 404 en JSON.

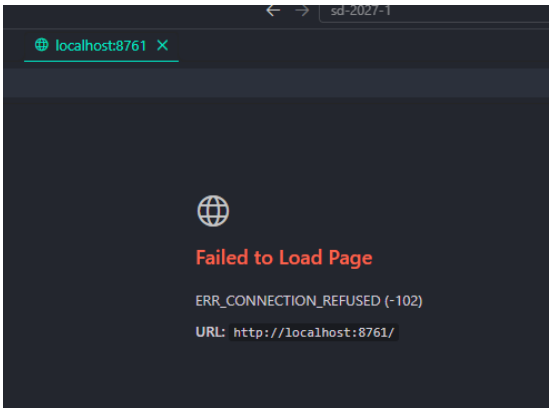


Figura 3: `localhost:8761` antes del paso 2: no carga, todavía no hay directorio.

2.1 a 2.4 · Cambios

Hora: 03:25

Para este paso hice los cambios que pide la guía para encender el directorio:

- En `discovery/.../DiscoveryApplication.java` agregué `@EnableEurekaServer` (con su `import`), que es la anotación que convierte la aplicación en un servidor Eureka.

- En `discovery/.../application.yml` puse `register-with-eureka: false` y `fetch-registry: false`, para que el directorio no se intente anotar en sí mismo.
- En los cinco `pom.xml` descomenté la dependencia del cliente de Eureka, que es la que hace que cada servicio se anote al arrancar y mande sus heartbeats.
- En el `docker-compose.yml` descomenté el servicio `discovery` (puerto 8761) y el `depends_on` hacia él en los cinco servicios.

Después reconstruí todo con `docker compose up -d -build`. La guía decía que esto tardaba unos 9 minutos, pero en mi caso tardó 2 minutos y medio (de 03:25:12 a 03:27:38), y esto fue porque las dependencias de Maven ya estaban en la caché de Docker desde la construcción que hice antes de la clase.

```
PS C:\Users\battl\Documents\sd-2027-1\entregas\calderon_victor\s07\mexi-banco> docker compose ps
NAME                                IMAGE                                COMMAND                  SERVICE         CREATED          STATUS                    PORTS
mexi-banco-v06-cuenta-1             mexi-banco-v06-cuenta                "java -jar app.jar"      cuenta          48 seconds ago   Up 40 seconds (healthy)   0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp
mexi-banco-v06-db-1                 postgres:17-alpine                   "docker-entrypoint.s…"   db              11 minutes ago   Up 11 minutes (healthy)   5432/tcp
mexi-banco-v06-discovery-1          mexi-banco-v06-discovery             "java -jar app.jar"      discovery       48 seconds ago   Up 46 seconds (healthy)   0.0.0.0:8761->8761/tcp, [::]:8761->8761/tcp
mexi-banco-v06-movimiento-1         mexi-banco-v06-movimiento            "java -jar app.jar"      movimiento      48 seconds ago   Up 40 seconds (healthy)   0.0.0.0:8082->8080/tcp, [::]:8082->8080/tcp
mexi-banco-v06-notificacion-1       mexi-banco-v06-notificacion          "java -jar app.jar"      notificacion    48 seconds ago   Up 39 seconds (healthy)   0.0.0.0:8084->8080/tcp, [::]:8084->8080/tcp
mexi-banco-v06-spei-1               mexi-banco-v06-spei                  "java -jar app.jar"      spei            48 seconds ago   Up 40 seconds (healthy)   0.0.0.0:8085->8080/tcp, [::]:8085->8080/tcp
mexi-banco-v06-transferencia-1      mexi-banco-v06-transferencia         "java -jar app.jar"      transferencia   48 seconds ago   Up 39 seconds (healthy)   0.0.0.0:8083->8080/tcp, [::]:8083->8080/tcp
PS C:\Users\battl\Documents\sd-2027-1\entregas\calderon_victor\s07\mexi-banco>
```

Figura 4: `docker compose ps` con el directorio: siete contenedores `healthy`.

2.5 · Comprobación

Hora: 03:28

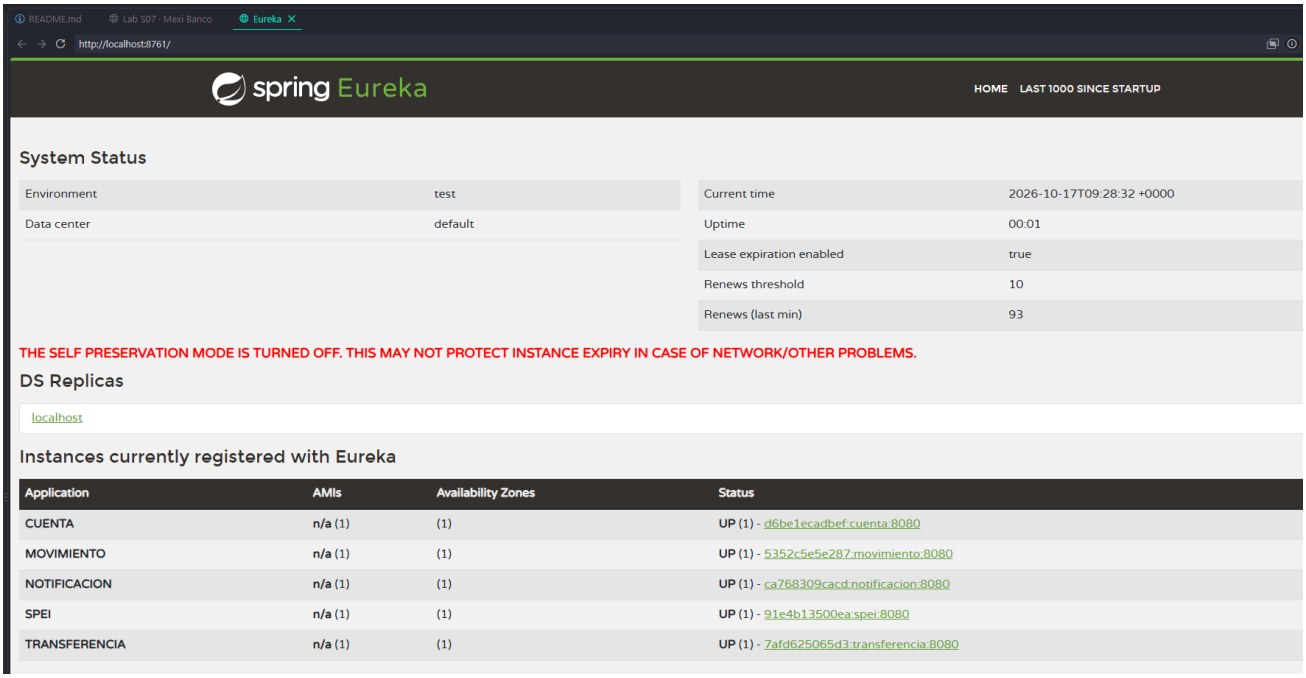


Figura 5: Tablero de Eureka: CUENTA, MOVIMIENTO, NOTIFICACION, SPEI y TRANSFERENCIA en `UP (1)`.

Ahora sí, `localhost:8761` ya carga y nos muestra el tablero de Eureka. En la tabla de "Instances currently registered with Eureka" aparecen los cinco servicios con `UP (1)`, es decir, una copia viva

de cada uno. Lo interesante aquí es que nadie los dio de alta a mano, sino que cada uno se anotó solo al arrancar con su `spring.application.name`. También podemos ver que el identificador de cada instancia empieza con el ID de su contenedor, por ejemplo, `d6be1ecadbef:cuenta:8080` corresponde al contenedor `d6be1ecadbef` (`mexi-banco-v06-cuenta-1` en `docker ps`). Por último, el letrero rojo de "THE SELF PRESERVATION MODE IS TURNED OFF" es normal, ya que en el compose se apagó la autopreservación para que Eureka borre rápido a los servicios que dejan de mandar heartbeats.

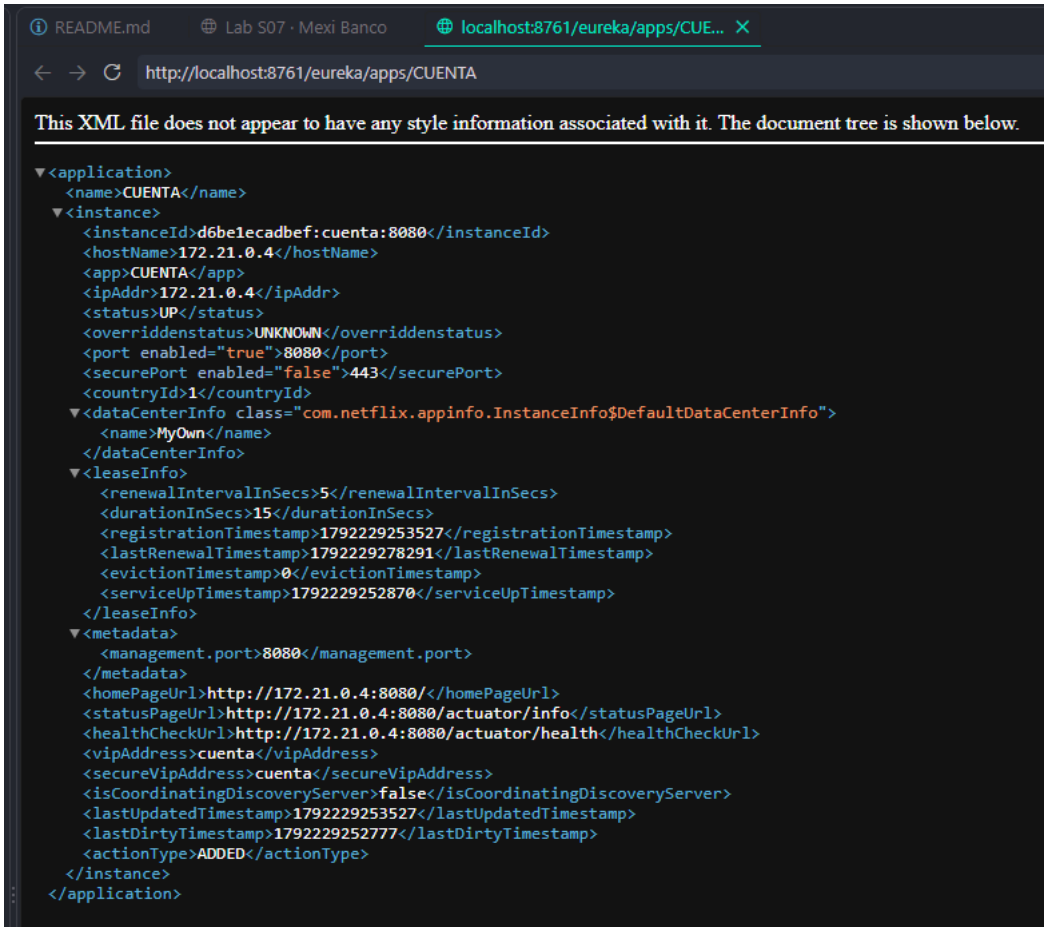


Figura 6: `localhost:8761/eureka/apps/CUENTA`: lo que el directorio sabe de `cuenta`.

Este XML es básicamente lo mismo que el tablero, pero pensado para las máquinas, es decir, es lo que recibe un servicio cuando le pregunta al directorio por `cuenta`. Aquí vemos que trae `hostName` e `ipAddr` con `172.21.0.4` (que es la IP del contenedor dentro de la red de Docker, no `localhost`), el `port` 8080, el `status` en `UP`, y dentro de `leaseInfo` los valores `renewalIntervalInSecs` 5 y `durationInSecs` 15. Estos últimos son justamente el heartbeat cada 5 s y el plazo de 15 s que se configuraron en el compose (`EUREKA_LATIDO_S` y `EUREKA_EXPIRA_S`).

Experimento corto: apagar y prender `notificacion`

Hora: 03:32

Aunque este experimento era opcional, decidí hacerlo para ver cómo reaccionaba el directorio. Apagué `notificacion` con `docker compose stop` y luego lo volví a prender con `docker compose start`. Además de estar recargando el tablero, dejé corriendo un ciclo que cada segundo le preguntaba a la API de Eureka (`/eureka/apps/NOTIFICACION`) por su estado, para tener las horas exactas:

```
03:32:09 NOTIFICACION "status":"UP"
         (03:32:52 docker compose stop)
03:32:56 contenedor exited
03:32:56 NOTIFICACION "status":"DOWN"
03:33:01 NOTIFICACION AUSENTE     <- ya no está en el registro (9 s)
         (03:34:00 docker compose start)
03:34:01 contenedor running
03:34:05 NOTIFICACION "status":"UP"   <- de vuelta (5 s)
```

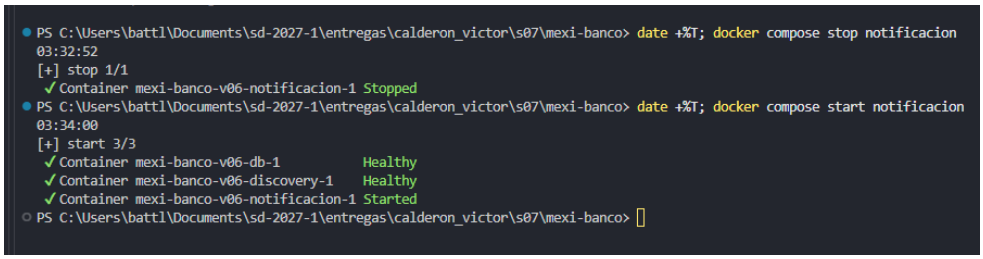


Figura 7: Los dos comandos con su hora: `stop` a las 03:32:52 y `start` a las 03:34:00.

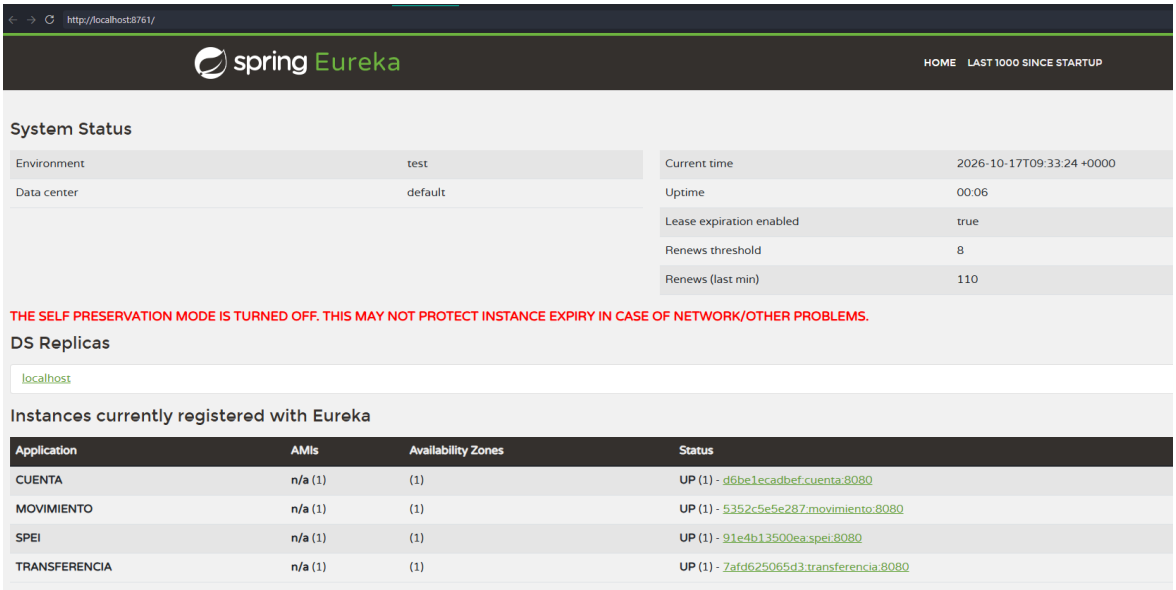


Figura 8: 03:33:24: el tablero ya no muestra NOTIFICACION.

¿Por qué no desaparece al instante? La guía dice que en el ensayo tardó unos 18 segundos en desaparecer, pero a mí me tomó solo 9 (de 03:32:52 a 03:33:01). Por lo que estuve revisando, la diferencia tiene que ver con la forma en que se apagó. Con `stop`, Docker manda un `SIGTERM` y Spring se apaga con calma, por lo que antes de morir el cliente de Eureka le avisa al directorio que se va (por eso, 4 s después del `stop`, primero aparece como `DOWN`). Los otros ∼5 s hasta que desaparece son lo que tarda en

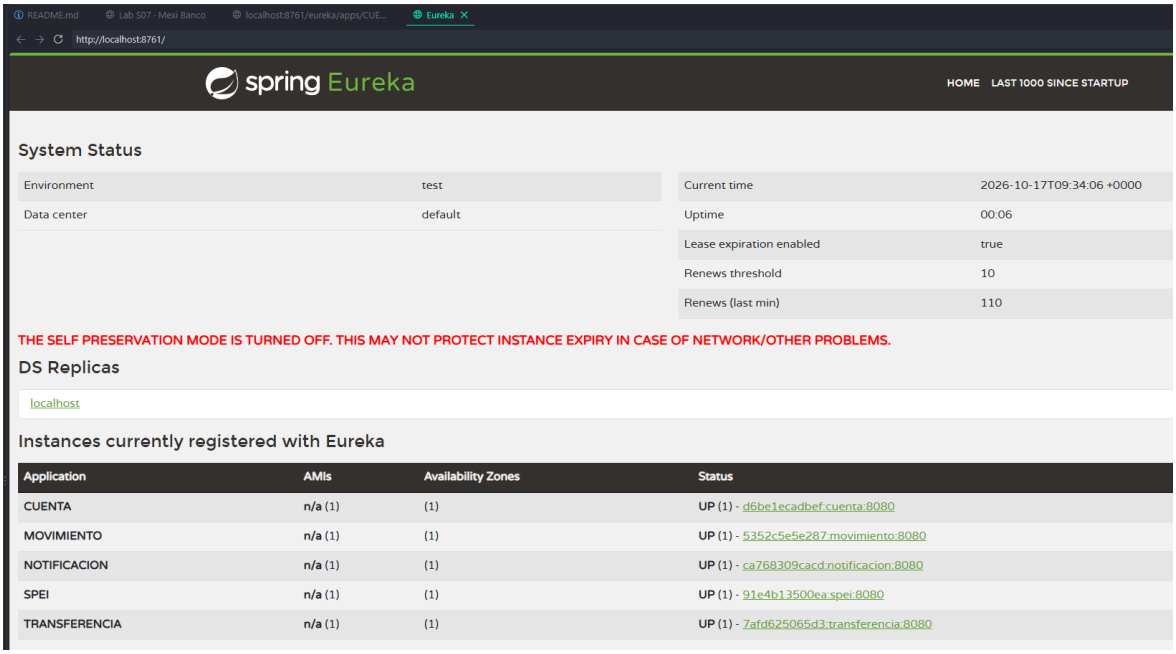


Figura 9: 03:34:06: NOTIFICACION regresa con el mismo identificador de contenedor.

refrescarse la copia que el directorio les entrega a los clientes (`EUREKA_CACHE_MS` = 5000). En cambio, si el servicio muere sin despedirse, el directorio solo se entera porque dejan de llegar los heartbeats, y entonces tiene que esperar el plazo de 15 s más su barrido periódico (cada 5 s), para no borrar a alguien que solo se tardó un poco. Esto último es justo lo que pasa con el `kill` del paso 5. Para el regreso, el servicio se volvió a anotar solo en cuanto arrancó: 5 s después del `start` ya estaba en `UP`, y con el mismo identificador `ca768309cacd`, ya que es el mismo contenedor.

Piensa

¿Alguien está usando el directorio para encontrar a `cuenta`? No. Aunque los cinco servicios ya están anotados en el directorio, nadie le está preguntando nada. Si revisamos el compose, `transferencia` y `spei` siguen llamando a `CUENTA_URL: http://cuenta:8080`, que es una dirección fija que resuelve el DNS interno de Docker. Es decir, el registro existe, pero por ahora los únicos que lo estamos leyendo somos nosotros en el tablero.

3. El gateway: una sola puerta

Antes del gateway: el 400 del 8082

Hora: 03:24

Para comprobar el problema de las puertas, hice una petición directa a `movimiento`. Registrar un movimiento es una operación interna, ya que solo la usan los servicios entre ellos, pero como el puerto 8082 está publicado, cualquiera la puede alcanzar desde afuera. Al mandar un `POST` vacío responde `400 Bad Request` (`"Failed to read request"`) y no un 404, lo que significa que el endpoint sí existe y está abierto, solo le faltaron datos.

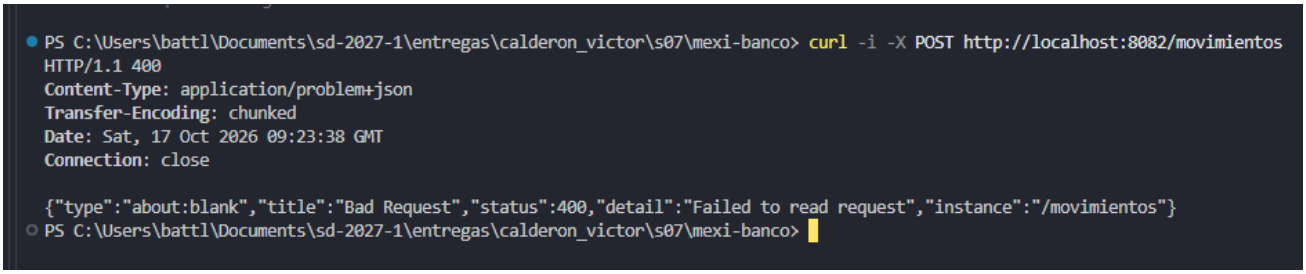


Figura 10: `POST localhost:8082/movimientos`: 400. El endpoint interno existe y cualquiera lo alcanza desde afuera.

3.1 y 3.2 · Rutas y gateway encendido

Hora: 03:35

En `gateway/src/main/resources/application.yml` agregué, debajo de la ruta de `cuenta` que ya venía como ejemplo, las rutas de `movimiento` (solo `GET`), `transferencia`, `notificacion` (solo `GET`) y `spei` (solo `POST`). Todas usan `uri: lb://<servicio>` y `StripPrefix=1`, y quedaron de la siguiente forma:

```
      - id: movimiento
        uri: lb://movimiento
        predicates:
          - Path=/api/movimientos
          - Method=GET
        filters:
          - StripPrefix=1
      - id: transferencia
        uri: lb://transferencia
        predicates:
          - Path=/api/transferencias,/api/transferencias/{id}
        filters:
          - StripPrefix=1
      - id: notificacion        # igual que movimiento, solo GET
      ...
      - id: spei                # Path=/api/spei, solo POST
      ...
```

Luego descomenté el servicio `gateway` en el compose y lo levanté con `docker compose up -d -build gateway` (de 03:34:57 a 03:35:15). Al recargar el tablero apareció un sexto renglón, ya que el gateway también es cliente de Eureka y se anota, porque necesita preguntarle al directorio para poder resolver `lb://cuenta`.

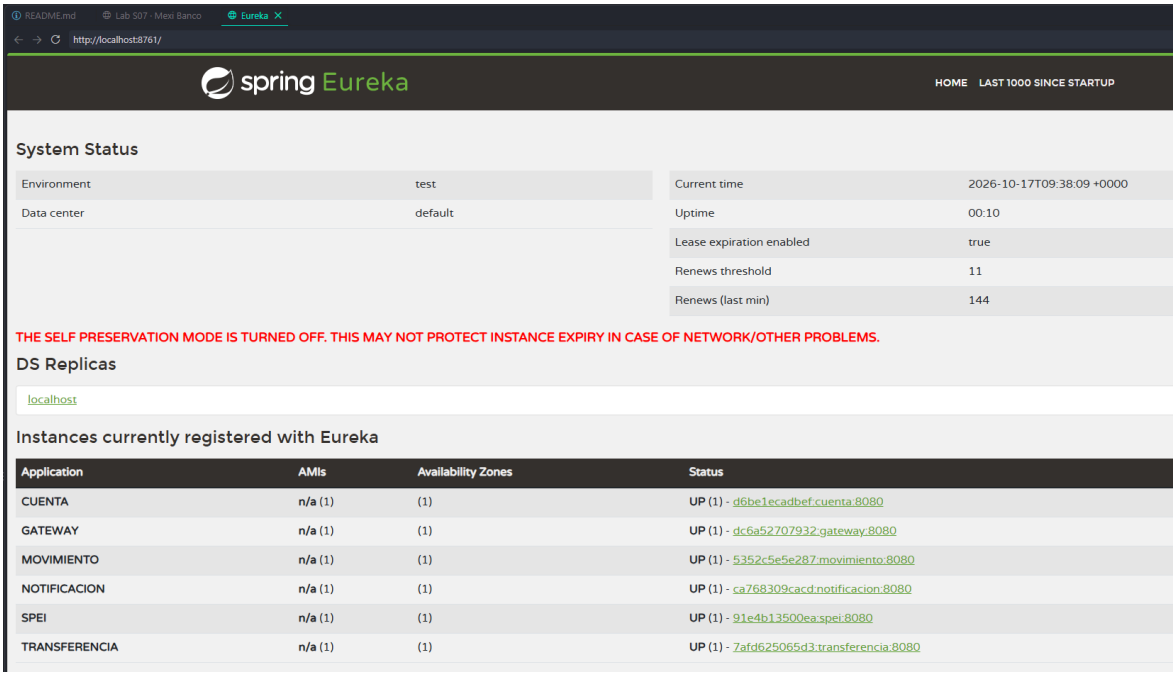


Figura 11: El tablero con un sexto renglón: GATEWAY también se anota.

3.3 · Pruebas por la puerta

Hora: 03:38

```
battl@Emiliano MINGW64 ~/Documents/sd-2027-1/entregas/calderon_victor/s07/mexi-banco (trabajo-s07)
$ curl -i http://localhost:8080/api/cuentas/000
HTTP/1.1 404 Not Found
transfer-encoding: chunked
X-Instancia: d6be1ecadbef
Content-Type: application/problem+json
Date: Sat, 17 Oct 2026 09:38:48 GMT

{"type":"about:blank","title":"Not Found","status":404,"detail":"No existe la CLABE 000","instance":"/cuentas/000"}
battl@Emiliano MINGW64 ~/Documents/sd-2027-1/entregas/calderon_victor/s07/mexi-banco (trabajo-s07)
$ curl -i -X POST http://localhost:8080/api/movimientos
HTTP/1.1 404 Not Found
Content-Type: application/json
Content-Length: 130

{"timestamp":"2026-10-17T09:38:57.672+00:00","path":"/api/movimientos","status":404,"error":"Not Found","requestId":"ffa73982-49"}
battl@Emiliano MINGW64 ~/Documents/sd-2027-1/entregas/calderon_victor/s07/mexi-banco (trabajo-s07)
$
```

Figura 12: (a) `GET /api/cuentas/000` y (b) `POST /api/movimientos` por el gateway: los dos dan 404.

¿Quién produjo cada 404? Aunque los dos dan 404, no vienen del mismo lugar:

- **(a) Lo produjo `cuenta`.** La respuesta trae la cabecera `X-Instancia: d6be1ecadbef`, que es el contenedor de `cuenta`, y además viene con el formato de errores de Mexi Banco (`"No existe la CLABE 000"`, con `instance` igual a `/cuentas/000`). Es decir, el gateway sí encontró la ruta, le preguntó al directorio dónde estaba `cuenta`, le quitó el `/api` y se la pasó, y fue `cuenta` quien dijo que esa CLABE no existe.
- **(b) Lo produjo el gateway.** Esta no trae `X-Instancia` y el cuerpo tiene otro formato (`timestamp`, `path`, `requestId`), que es el de los errores de Spring Cloud Gateway. Como ninguna ruta acepta un `POST` a `/api/movimientos` (la de `movimiento` solo acepta `GET`), la petición nunca

salió del gateway. Por lo que la misma operación que en el 8082 nos daba 400 ya no se puede alcanzar por la puerta.



Figura 13: Alta de mi cuenta `S071792229979` por el gateway (201 Created) y su consulta (200 OK, saldo 0.00).

Por último, creé mi propia cuenta por el gateway. Quedó con la CLABE `S071792229979` (que es `"S07"` más la hora en segundos), y su liga por el gateway es `http://localhost:8080/api/cuentas/S071792229979`. Las dos respuestas traen `X-Instancia: d6be1ecadbef`, ya que en ese momento todavía había una sola copia de `cuenta`.

4. Tres copias de cuenta y una que se muere

4.1 · Tres copias

Hora: 03:41

Para este paso, en el compose cambié los `ports` de `cuenta` por `deploy: replicas: 3`, y también borré los `ports` de `movimiento`, `transferencia`, `notificacion` y `spei`, por lo que solo quedaron publicados el gateway (8080) y el directorio (8761). Después de eso corrí `docker compose up -d`.

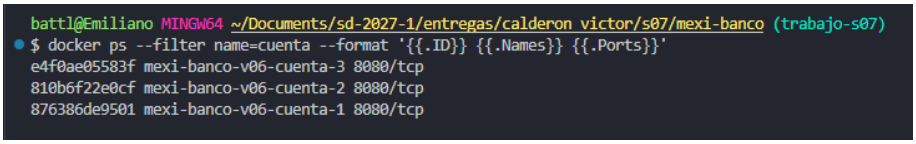


Figura 14: Las tres copias de `cuenta`: cada una con su ID y con `8080/tcp` sin flecha (escuchan por dentro, no están publicadas).

Los tres ID quedaron de la siguiente forma: `876386de9501` (`cuenta-1`), `810b6f22e0cf` (`cuenta-2`) y `e4f0ae05583f` (`cuenta-3`). Algo que noté es que la `cuenta-1` ya no era `d6be1ecadbef`, y esto es porque al cambiarle los puertos Compose la volvió a crear (lo mismo pasó con los otros cuatro servicios,

que también cambiaron de ID en el tablero). Por lo mismo, durante unos segundos Eureka mostró cuatro instancias de `cuenta`, con la vieja en `DOWN`, hasta que a las 03:41:36 ya quedó en tres.

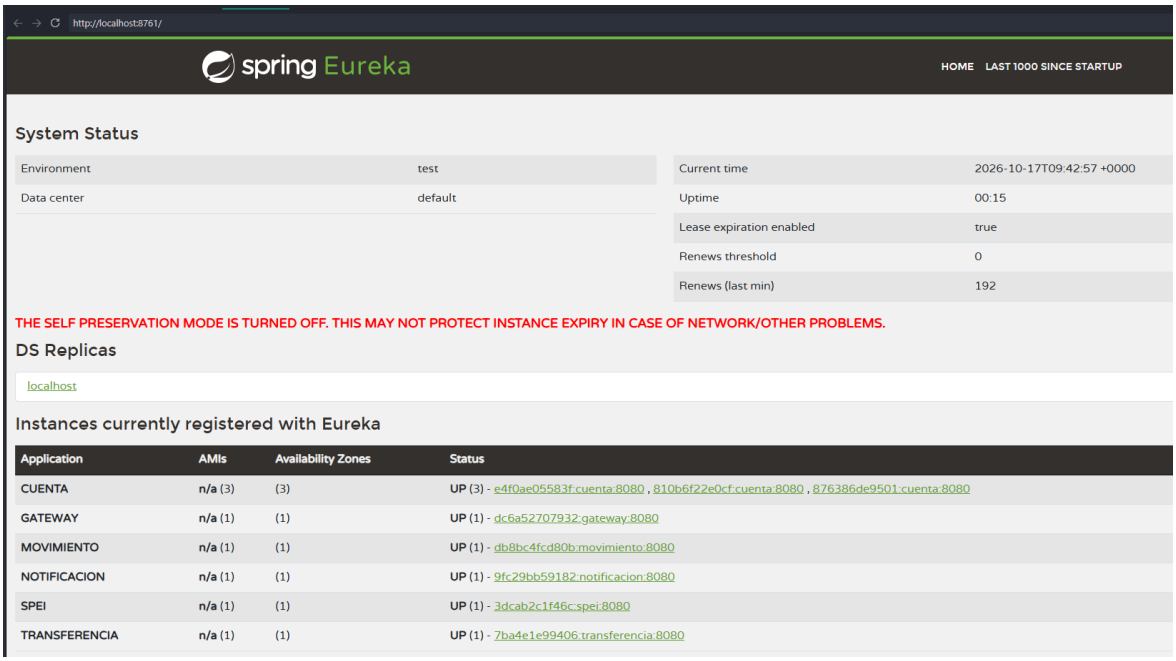


Figura 15: Tablero: CUENTA en `UP (3)` con los mismos tres ID de `docker ps`.

Algo importante aquí es que nadie le avisó al gateway que ahora hay tres copias, sino que lo sabe porque las tres se anotaron solas en el directorio, y el gateway le pregunta a este.

4.2 · Cómo reparte

Hora: 03:44

```
battl@Emiliano MINGW64 ~/Documents/sd-2027-1/entregas/calderon_victor/s07/mexi-banco (trabajo-s07)
$ for i in {1..9}; do
  curl -sS -D - -o /dev/null "$URL" | grep -i '^X-Instancia'
done
X-Instancia: 810b6f22e0cf
X-Instancia: e4f0ae05583f
X-Instancia: 876386de9501
X-Instancia: 810b6f22e0cf
X-Instancia: e4f0ae05583f
X-Instancia: 876386de9501
X-Instancia: 810b6f22e0cf
X-Instancia: e4f0ae05583f
X-Instancia: 876386de9501
```

Figura 16: Nueve consultas por el gateway: `X-Instancia` se turna entre las tres copias.

Como podemos ver, las nueve respuestas siguen el ciclo `810b6f22e0cf`, `e4f0ae05583f`, `876386de9501`, repetido tres veces (A, B, C, A, B, C, A, B, C), y son los mismos ID que nos dio `docker ps`. A este reparto por turnos se le conoce como “round robin”, y lo hace el balanceador del gateway.

En los logs podemos ver lo mismo, pero ahora desde el lado de `cuenta`: en la última ráfaga (09:45:59 UTC) las líneas van `cuenta-3`, `cuenta-2`, `cuenta-1`, `cuenta-3`, `cuenta-2`, `cuenta-1`…, cada una con `GET /cuentas/S071792229979 atendida por <su ID>`.

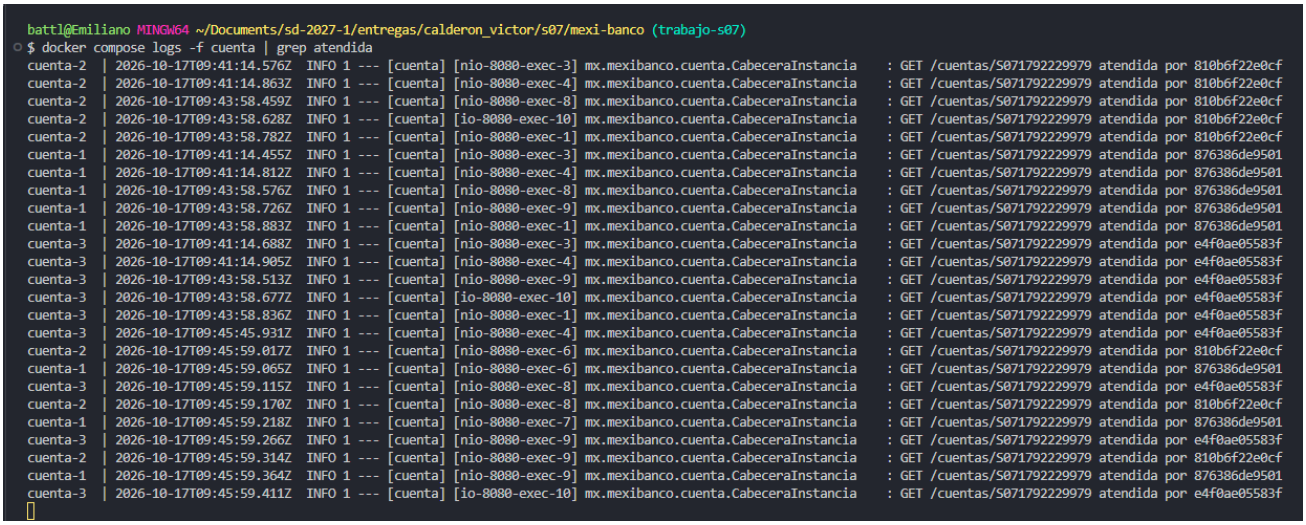


Figura 17: `docker compose logs -f cuenta | grep atendida`: desde el lado de `cuenta`, cada copia atiende una petición por turno.

Piensa

¿Cuando `transferencia` llama a `cuenta` por dentro, también reparte? No. El reparto que vimos aquí lo hace el balanceador del gateway, que resuelve `lb://cuenta` con la lista del directorio. Mientras tanto, `transferencia` sigue usando `http://cuenta:8080`, y ese nombre lo resuelve el DNS de Docker, no Eureka, por lo que `transferencia` no sabe cuántas copias hay ni cuáles están vivas. Esto es justo lo que se arregla en la tarea.

5 · ¿Y si una copia se muere?

Hora: 03:46

Mi predicción (antes de correrlo). Si el reparto funciona como debe, van a fallar algunas consultas, no todas: solo las que por turno le toquen a la copia muerta, y durante el tiempo que el sistema siga creyendo que esa copia está viva.

Para el experimento usé dos terminales. En la terminal 1 dejé corriendo una consulta por segundo a mi cuenta (60 consultas en total), y en la terminal 2 maté `cuenta-1` de golpe con `docker kill`. Mientras tanto estuve recargando el tablero, y además dejé un ciclo que cada segundo le preguntaba a la API de Eureka qué copias de `cuenta` tenía registradas.

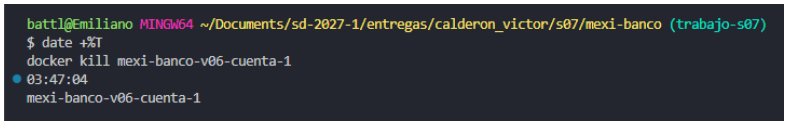


Figura 18: Terminal 2: `docker kill mexi-banco-v06-cuenta-1` a las 03:47:04.

03:46:49	200	03:47:06	500	03:47:32	500	03:47:53	200
03:46:50	200	03:47:09	200	03:47:35	200	03:47:54	200
03:46:51	200	03:47:10	200	03:47:36	200	03:47:55	200
03:46:53	200	03:47:11	500	03:47:38	200	03:47:56	200
03:46:54	200	03:47:14	200	03:47:39	500	03:47:57	200
03:46:55	200	03:47:15	200	03:47:42	200	03:47:58	200
03:46:56	200	03:47:16	200	03:47:43	200	03:47:59	200
03:46:57	200	03:47:17	500	03:47:44	200	03:48:00	200
03:46:58	200	03:47:21	200	03:47:45	200	03:48:01	200
03:46:59	200	03:47:22	200	03:47:46	200	03:48:03	200
03:47:00	200	03:47:23	500	03:47:47	200	03:48:04	200
03:47:01	200	03:47:26	200	03:47:48	200	03:48:05	200
03:47:02	200	03:47:27	200	03:47:49	200	03:48:06	200
03:47:04	200	03:47:28	200	03:47:50	200	03:48:07	200
03:47:05	200	03:47:29	500	03:47:52	200	03:48:08	200

Figura 19: Terminal 1: las 60 consultas (hora y código HTTP) durante la caída, guardadas con `tee`.

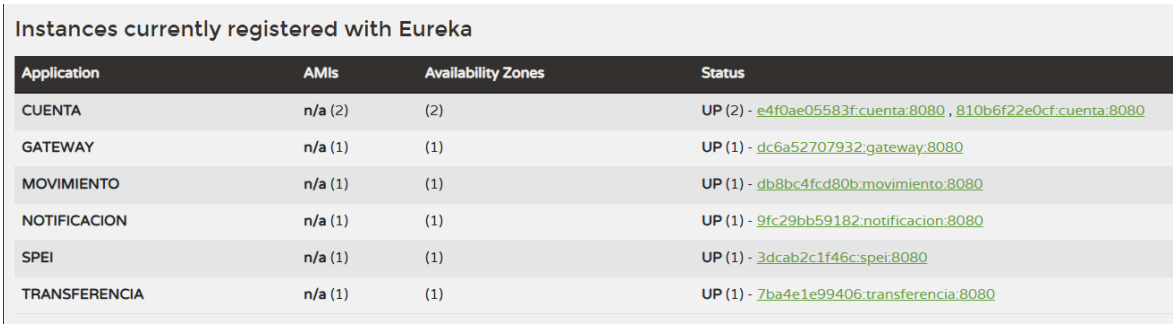


Figura 20: Tablero: CUENTA en `UP (2)`; la copia `876386de9501` ya salió del directorio.

`docker kill` de `cuenta-1`	03:47:04	
Primer error (500)	03:47:06	2 s después del kill
Eureka la saca: CUENTA pasa a `UP (2)`	03:47:35	31 s después del kill
Último error (500)	03:47:39	4 s después de salir del directorio
Errores vistos	7 de 60	todos 500
`docker start` de `cuenta-1`	03:48:51	
CUENTA regresa a `UP (3)`	03:49:00	9 s después del start

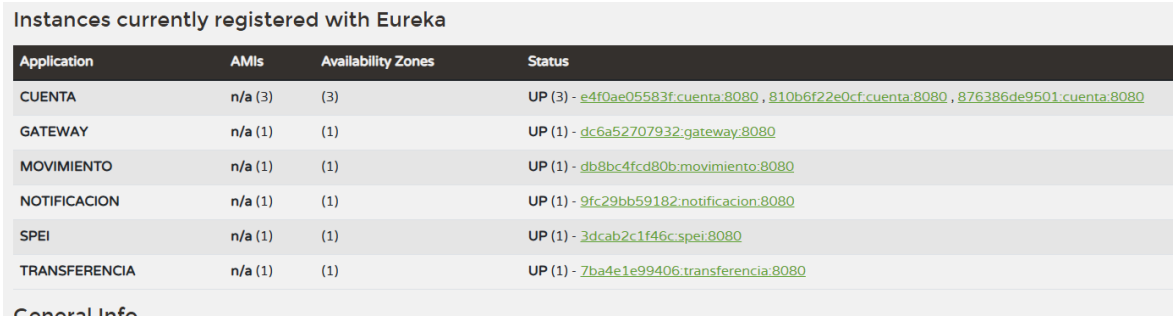
Cuadro 1: Lo que medí durante la caída.

¿Qué pasó y por qué? Mi predicción sí se cumplió en lo principal, ya que no fallaron todas, solo 7 de 60, y salteadas, más o menos una de cada tres (`200, 200, 500`). Estas eran las peticiones a las que, por turno, les tocaba la copia muerta, mientras que las otras dos copias siguieron contestando 200 todo el tiempo. También noté que cada consulta fallida tardaba unos 3 segundos en volver (por eso hay huecos en las horas, como de 03:47:06 a 03:47:09), y esto es porque el gateway intentaba conectarse a la IP de un contenedor que ya no existía y esperaba su `connect-timeout` de 2 s antes de rendirse con un 500. Solo hubo un par de errores seguidos (03:47:29 y 03:47:32), que creo que fue porque al refrescar su lista el balanceador reacomodó el turno.

Lo que no tenía previsto es que los errores siguieran después de que la copia salió del directorio, ya que Eureka la sacó a las 03:47:35 y el último 500 fue a las 03:47:39. Esto se debe a que nadie avisa de la caída, por lo que el sistema se va enterando por partes. Primero, el directorio no sabe que la copia murió, solo nota que dejaron de llegar los heartbeats, por lo que espera el plazo de 15 s para no borrar a alguien que solo se tardó, más su barrido periódico (cada 5 s) y el refresco de su caché (5 s); en total tardó 31 s. Después, el gateway tampoco ve el directorio en vivo, sino que guarda una copia de la lista que renueva cada 5 s, y su balanceador tiene encima otra caché de 5 s. Es por eso que le siguió mandando peticiones a la copia muerta unos segundos más, hasta las 03:47:39.

En total hubo errores intermitentes durante 33 s (de 03:47:06 a 03:47:39), lo cual es muy parecido a lo que dice la guía (de 20 a 30 s, y 7 errores en 45 consultas en el ensayo del profesor). Si hubiéramos dejado los valores por defecto de Spring (heartbeat de 30 s y plazo de 90 s), esto habría tardado minutos.

Si lo comparamos con el experimento del paso 2, la diferencia es bastante clara: `notificacion`, que se apagó con `stop`, se despidió y salió del directorio en 9 s, mientras que `cuenta-1`, que se cortó con `kill`, no pudo despedirse y tardó 31 s en salir.


Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
CUENTA	n/a (3)	(3)	UP (3) - e4f0ae05583f:cuenta:8080 , 810b6f22e0cf:cuenta:8080 , 876386de9501:cuenta:8080
GATEWAY	n/a (1)	(1)	UP (1) - dc6a52707932:gateway:8080
MOVIMIENTO	n/a (1)	(1)	UP (1) - db8bc4fcd80b:movimiento:8080
NOTIFICACION	n/a (1)	(1)	UP (1) - 9fc29bb59182:notificacion:8080
SPEI	n/a (1)	(1)	UP (1) - 3dcab2c1f46c:spei:8080
TRANSFERENCIA	n/a (1)	(1)	UP (1) - 7ba4e1e99406:transferencia:8080

General Info

Figura 21: Después de `docker start mexi-banco-v06-cuenta-1`: CUENTA regresa a `UP (3)` con el mismo ID `876386de9501`.

Por último, al revivirla con `docker start` (03:48:51) regresó con el mismo ID, ya que es el mismo contenedor, y se volvió a anotar sola, por lo que a las 03:49:00 el directorio ya mostraba otra vez `UP (3)`.

5. Cierre

Hora: 03:50

Para cerrar, estas son mis cuatro frases sobre lo que construimos en el laboratorio:

- **El directorio resuelve** el problema de saber qué servicios existen y en dónde está cada copia, sin que nadie tenga que escribir las direcciones a mano, ya que cada servicio se anota solo al arrancar y sale cuando deja de mandar heartbeats.
- **El gateway resuelve** que el cliente solo tenga una puerta (el 8080) en lugar de cinco, y que solo pase lo que tiene una ruta, por lo que las operaciones internas, como registrar un movimiento, ya no se pueden alcanzar desde afuera.
- **El balanceador resuelve** cómo repartir las peticiones entre varias copias del mismo servicio (por turnos, con round robin) y dejar de mandarle a las que ya no están en la lista del directorio.
- **Cuando murió una copia vi** 7 errores 500 salteados en 60 consultas durante unos 33 segundos, **porque** nadie avisó de la caída: el directorio tardó 31 s en notar que faltaban los heartbeats y sacarla, y el gateway todavía tardó unos segundos más en actualizar su lista, por lo que mientras tanto, por turno, una de cada tres peticiones le tocaba a la copia muerta.

6. Tarea: que transferencia también reparta

T.1 y T.2 · Cambios

Hora: 03:49

Hasta el paso 5, el único que repartía era el gateway, ya que por dentro `transferencia` seguía llamando a `http://cuenta:8080`, que es un nombre que resuelve el DNS de Docker y no el directorio. Para que también repartiera, hice dos cambios:

1. En `transferencia/.../clientes/ClientesHttp.java` agregué un `RestClient.Builder` con la anotación `@LoadBalanced`:

```
import org.springframework.boot.autoconfigure.web.client.RestClientBuilderConfigurer;
import org.springframework.cloud.client.loadbalancer.LoadBalanced;
...
@Bean
@LoadBalanced
RestClient.Builder restClientBalanceado(RestClientBuilderConfigurer configurador) {
    return configurador.configure(RestClient.builder());
}
```

   Lo que hace `@LoadBalanced` es que en cada petición toma el host de la URL (`cuenta`) como el nombre del servicio, entonces el balanceador le pregunta al directorio qué copias hay y lo cambia por la IP y el puerto de una de ellas. Por otro lado, el `configurador` le aplica lo mismo que Spring Boot le pone a su builder, incluidos los tiempos máximos (`tiemposMaximos`).

2. En el `docker-compose.yml` cambié las URLs de `transferencia` para que quedaran solo con el nombre, sin el puerto:

```
      CUENTA_URL: http://cuenta
      MOVIMIENTO_URL: http://movimiento
      NOTIFICACION_URL: http://notificacion
```

Después reconstruí solo `transferencia` con `docker compose up -d -build transferencia` (de 03:49:55 a 03:50:18).



Figura 22: `docker compose ps` después de la tarea: todo `healthy`, solo el gateway (8080) y el directorio (8761) publicados, y `transferencia` recién creada.

T.3 · Prueba con `./demo-v06.sh`

Hora: 03:52

La demo corrió completa desde el primer intento, sin ningún timeout, y todo pasó por el gateway (`localhost:8080`):

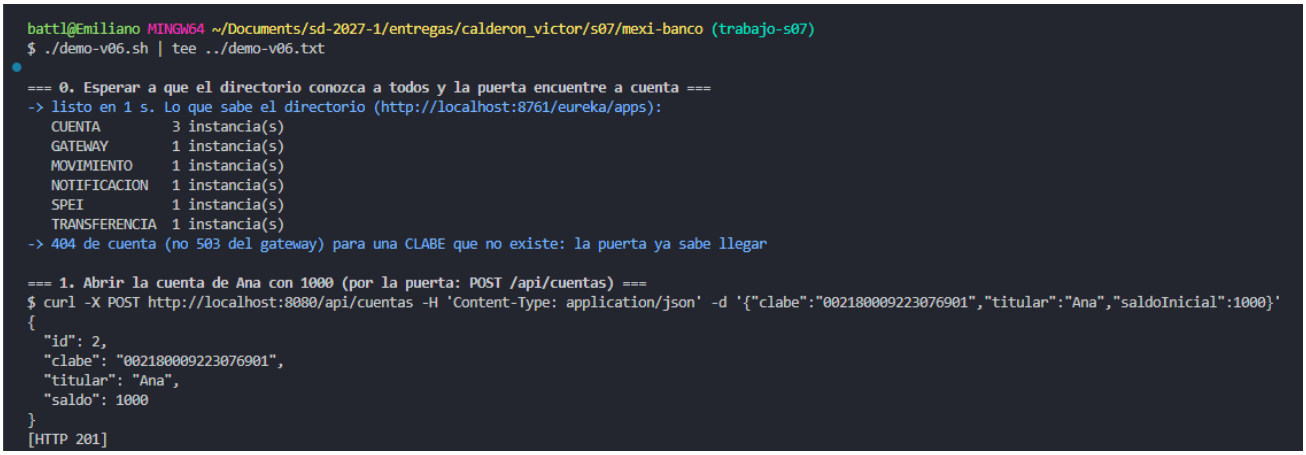


Figura 23: `./demo-v06.sh`, paso 0 y 1: el directorio conoce a los seis (CUENTA con 3 instancias) y se abre la cuenta de Ana con 1000.

Momento	Ana	Beto	Comentario
Apertura	1000.00	500.00	suman 1500
Transferencia de 200	800.00	700.00	suman 1500: no se crea ni se pierde
SPEI de 50 (enviado dos veces)	750.00	700.00	cobrado una vez (`id` 1 → 1)
Errores 404 y 422	750.00	700.00	sin cambios

Cuadro 2: Saldos de la demo: coinciden con lo esperado (800/700 y luego 750/700).



Figura 24: Pasos 2 y 3: cuenta de Beto con 500 y transferencia de 200. `transferencia` llamó a `http://cuenta` y sus cuatro llamadas se repartieron entre las tres copias.



Figura 25: Pasos 4 y 5: la transferencia registrada y los saldos: Ana 800, Beto 700.

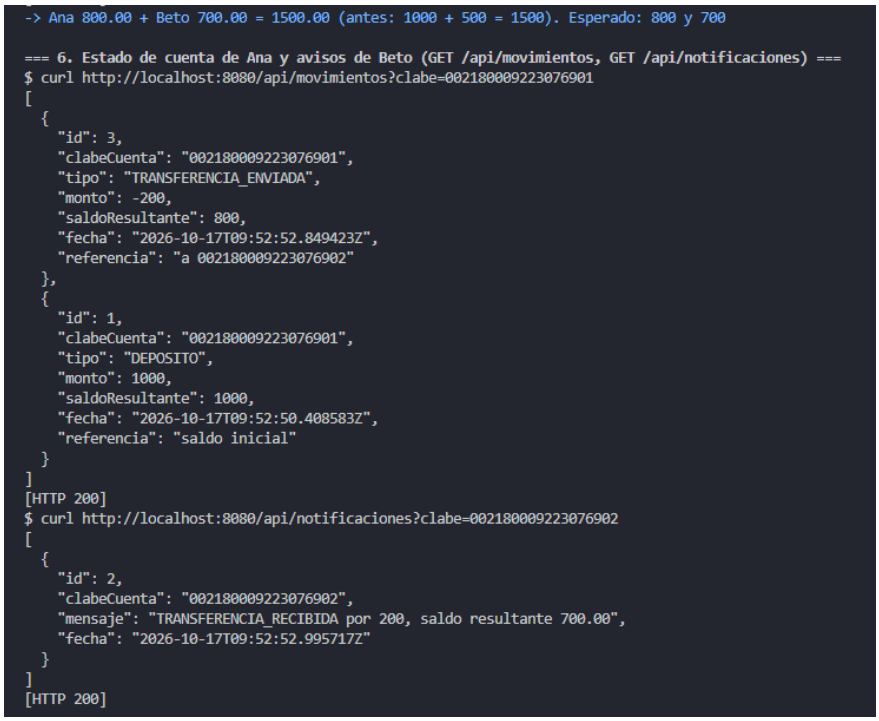


Figura 26: Paso 6: estado de cuenta de Ana y aviso de Beto, por el gateway (`GET /api/movimientos` y `GET /api/notificaciones`).

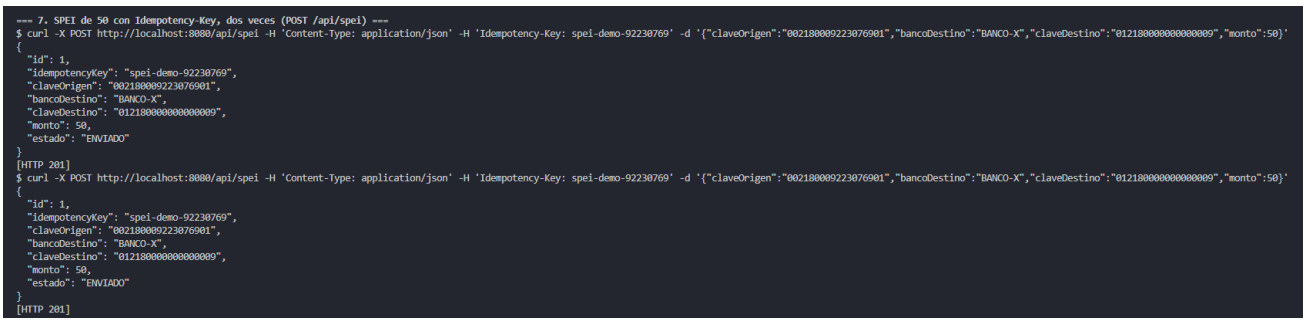


Figura 27: Paso 7: el SPEI de 50 enviado dos veces con la misma `Idempotency-Key`; las dos respuestas traen el mismo `id` 1.

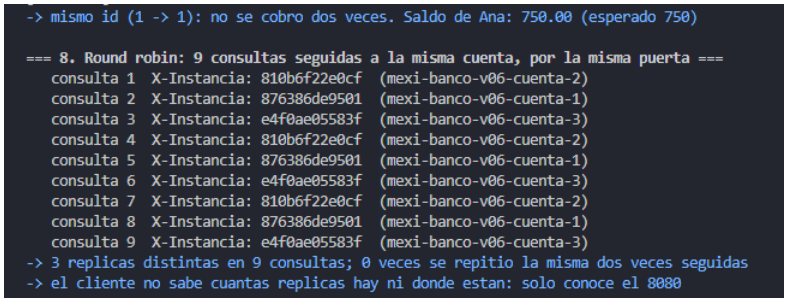


Figura 28: Paso 7 y 8: no se cobró dos veces (Ana queda en 750) y el round robin de las nueve consultas.

```
=== 9. Los endpoints internos no tienen puerta ===
$ curl -X POST http://localhost:8080/api/cuentas/00218000922307690l/abonos -H 'Content-Type: application/json' -d '{"monto":1000000}'
{
  "timestamp": "2026-10-17T09:52:58.248+00:00",
  "path": "/api/cuentas/002180009223076901/abonos",
  "status": 404,
  "error": "Not Found",
  "requestId": "a441dd30-328"
}
[HTTP 404]
-> 404: el gateway no tiene ruta para /cargos ni /abonos. Adentro de la red siguen abiertos (seguridad: mas adelante)
```

Figura 29: Paso 9: `POST /api/cuentas/.../abonos` por la puerta da 404: los endpoints internos no tienen ruta.

```
=== 10. Errores de negocio, por la puerta, con el mismo codigo que dio cuenta ===
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":"002180009223076901","claveDestino":"002180009223076999","monto":10}'
{
  "type": "about:blank",
  "title": "Not Found",
  "status": 404,
  "detail": "No existe la CLABE 002180009223076999",
  "instance": "/transferencias",
  "servicio": "cuenta"
}
[HTTP 404]
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":"002180009223076901","claveDestino":"002180009223076902","monto":99999}'
{
  "type": "about:blank",
  "title": "Unprocessable Entity",
  "status": 422,
  "detail": "Saldo insuficiente en la CLABE 002180009223076901",
  "instance": "/transferencias",
  "servicio": "cuenta"
}
[HTTP 422]

=== 11. Los saldos no se movieron con los errores ===
-> Ana 750.00 (esperado 750) + Beto 700.00 (esperado 700) = 1450.00; SPEI se llevo 50 a otro banco

Fin. Para ver cuanto tarda el directorio en enterarse de una caida: ./demo-v06.sh --latido
```

Figura 30: Pasos 10 y 11: errores de negocio (404 y 422, con `"servicio":"cuenta"`) y los saldos finales 750/700 sin cambios.

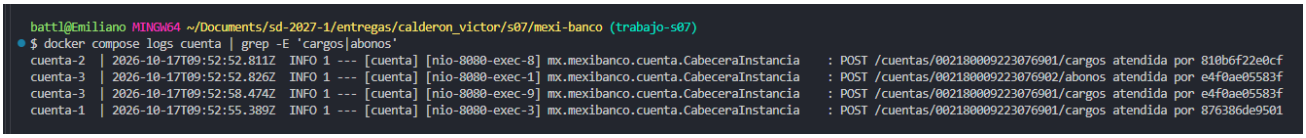


Figura 31: `docker compose logs cuenta | grep -E 'cargos|abonos'`: qué copia atendió cada cargo y cada abono.

¿Qué muestran los logs? Si las ordenamos por hora, las líneas son:

- 09:52:52.811, `cuenta-2`: el cargo de la transferencia (a Ana).
- 09:52:52.826, `cuenta-3`: el abono de esa misma transferencia (a Beto).
- 09:52:55.389, `cuenta-1`: el cargo del SPEI.
- 09:52:58.474, `cuenta-3`: el intento de cargo por 99 999, que `cuenta` rechazó con 422 (saldo insuficiente).

Como podemos ver, el cargo y el abono de una misma transferencia cayeron en copias distintas, con solo 15 ms de diferencia, y en la salida de la demo también se ve que las dos consultas previas (origen y destino) se repartieron (`cuenta-3` y `cuenta-1`). Es decir, ahora quien decide a qué copia va cada llamada es el balanceador de `transferencia`, con la lista del directorio, y ya no el DNS de Docker, por lo que si una copia muere, sale de su lista en cuanto el directorio la borra. Algo que vale la pena aclarar es que el hecho de que el cargo y el abono se atiendan en procesos distintos no cambia nada para el dinero, ya que las tres copias comparten la misma base de `cuenta`.

Por último, algo que noté es que el SPEI no forma parte de la tarea, y `spei` sigue llamando a `http://cuenta:8080`. Su cargo también cayó en una de las copias, pero esa la eligió el DNS de Docker (que responde con las IP de las tres réplicas) y no el directorio, por lo que si una copia muriera, `spei` no se enteraría.